Roteiro de fala, slide a slide

O que dizer em cada slide, com o relógio ao lado. Não é para decorar — é para ensaiar em voz alta duas vezes e depois esquecer o texto e lembrar só da ordem.

COMO USAR ISTO

O texto em preto é o que você fala. Não precisa ser palavra por palavra — precisa ser a mesma ideia, na mesma ordem. As palavras em *laranja* são as que você **não pode deixar de dizer**: são termos que a banca procura na ficha de avaliação.

As caixas verdes são o que você faz — apertar tecla, apontar, esperar resposta, rodar código.

Ensaie de pé e em voz alta. Ler por dentro engana: você lê 40% mais rápido do que consegue falar.

Parte 1 — Plano de aula

5 minutos · Art. 40, I · **não inicie o cronômetro ainda**

01 Capa

Identificação

`0:00 — 0:45`

"Boa noite. Eu sou Guilherme Santos Rabelo, candidato à instrutoria da Trilha Básica.

O tema que me foi sorteado é **Lógica de Programação e Versionamento**.

Este é o plano de aula impresso — estou entregando agora, antes de começar.

Vou usar cinco minutos para apresentar o plano e depois vinte para dar a aula em si."

FAÇA Entregue as 3 cópias impressas **antes** de falar qualquer outra coisa. Fique de pé. Não leia a capa em voz alta — a banca já está lendo.

02 Problema gerador e objetivos

O slide que mais vale ponto no plano

0:45 – 2:00

"O plano parte de um **problema gerador**, não de uma lista de conteúdos. É o que a metodologia de **Aprendizagem Baseada em Projetos** exige.

O problema é este: como transformar uma lista bagunçada de 120 vagas de emprego na resposta 'estas aqui são pra mim' — e construir isso em squad, sem ninguém sobrescrever o trabalho do outro.

Reparem que a pergunta tem duas metades. A primeira é lógica: transformar dado em decisão. A segunda é versionamento: construir junto sem destruir. São exatamente as duas metades do nome do módulo.

E os objetivos estão todos escritos com verbo observável: decompor, construir, reconhecer, abrir, versionar, colaborar. Eu preciso conseguir **verificar** se o aluno faz — não se ele 'entendeu'."

ATENÇÃO Aqui estão 15 dos 30 pontos do plano: clareza dos objetivos (5) e aplicação do PBL (10). Diga "problema gerador" e "Aprendizagem Baseada em Projetos" com todas as letras.

03 Competências e método

Técnicas, comportamentais e o ciclo PBL

2:00 – 3:10

"As competências não são só técnicas. Como a entrega é em squad e passa por Pull Request, o aluno exercita revisão por pares, comunicação técnica escrita e resiliência ao erro — que é o que o mercado cobra e que não se mede em prova.

A metodologia é o ciclo de **Aprendizagem Baseada em Projetos**, em cinco etapas: imersão no problema, investigação, construção, entrega e socialização.

A aula de hoje percorre as cinco. E o mesmo ciclo se repete, em escala maior, ao longo das 40 horas do módulo."

FAÇA Diga os nomes das cinco etapas em voz alta, apontando cada caixa. É rápido e a banca marca.

04 Cronograma e recursos

Mostra que você leu o Art. 40

3:10 – 4:10

"O cronograma cabe nos **vinte minutos de exposição** previstos no edital: três minutos de imersão no problema, três de investigação, sete de construção do código, dois de abertura de projeto, quatro de versionamento e um de fechamento.

Os recursos: notebook e TV, editor e terminal para rodar o código ao vivo, um repositório-modelo no GitHub, o conjunto de dados em JSON, e o quadro — porque o algoritmo é escrito em português antes de virar código."

> **ATENÇÃO** Dizer "vinte minutos de exposição" já sinaliza domínio das regras e ajuda na nota de gestão do tempo.

05 Avaliação e indicadores

Última do plano — saia daqui aos 5:00

4:10 – 5:00

"A avaliação é formativa e por produto. Não tem prova escrita: o repositório do aluno é a evidência do aprendizado, e eu acompanho o progresso pelos commits, não só pela entrega final.

E eu monitoro com números, não com impressão: cem por cento dos alunos com repositório criado, no mínimo três commits com mensagem descritiva por aluno, oitenta por cento dos algoritmos rodando sem erro.

Esse é o plano. Vou para a aula."

> **SE ATRASOU** Aos 5:00 você tem que estar saindo daqui. Se estourou, pule direto para a aula: ela vale 60 pontos e o plano, que vale 30, já está impresso na mão deles.

Parte 2 — A aula

20 minutos · Art. 40, II · **aperte T agora para iniciar o cronômetro**

06 O gancho

Contato visual, sem olhar a tela

`0:00 — 0:40`

"Vamos começar com uma situação de verdade.

Vocês entraram no Jovem Tech para trabalhar com tecnologia. Daqui a alguns meses, vocês vão abrir um site de vagas e encontrar 120 anúncios de uma vez.

Quantos desses são realmente pra vocês? Cinco? Dez?

E como descobrir isso **sem ler os 120 na mão**?"

FAÇA Aperte `T`. Fale de pé, olhando para a banca. Este slide é 100% contato visual.

07 A lista bagunçada

Plante a ideia de padrão

`0:40 — 1:30`

"Este é o arquivo com as vagas. Não precisa entender cada linha agora.

Só quero que vocês reparem numa coisa: toda vaga tem exatamente os mesmos quatro campos. Título, empresa, nível e salário. Sempre os mesmos.

Guardem isso, porque é a chave de tudo que vem depois."

FAÇA Aponte para os campos repetidos nos dois blocos. Você vai colher isso no slide 09.

08 O problema gerador

O slide mais importante da aula

1:30 – 2:40

"Esta é a pergunta que a gente vai responder hoje. É o nosso **problema gerador**.

Como transformar 120 vagas bagunçadas na resposta 'estas aqui são pra mim' — e construir isso em equipe, sem ninguém apagar o trabalho do outro.

(pausa de dois segundos — deixe a pergunta no ar)

Reparem que ela tem duas metades. A primeira: como o computador decide o que serve. Isso é **lógica de programação**.

A segunda: como a gente constrói junto sem destruir. Isso é **versionamento**.

E olha só — é exatamente o nome do nosso módulo."

ATENÇÃO Pause de verdade depois de ler a pergunta. O silêncio faz a banca registrar que aquilo é o eixo da aula.

09 Pensamento computacional

Primeira interação com a turma

2:40 – 3:50

"Antes de escrever qualquer código, a gente pensa. Programar não começa no teclado.

São quatro passos. **Decompor**: quebrar o problema grande em pedaços pequenos. 'Achar minhas vagas' vira: ler a lista, olhar uma por uma, separar as que servem.

Padrões: lembram que toda vaga tinha os mesmos campos? Aquilo é padrão. O que serve pra uma serve pras 120.

Abstração: jogar fora o que não importa. Deixa eu perguntar pra vocês — nessa vaga aí, o que dá pra jogar fora?"

(espere a resposta de verdade — não responda por eles)

"Isso. E o que sobra é nível e salário. E aí sim, o quarto passo: escrever o **algoritmo**, o passo a passo."

FAÇA Esta é a interação nº 1 das duas obrigatórias (vale 5 pontos). Espere alguém responder e aponte para quem respondeu.

10 A receita em português

Momento de maior clareza didática

3:50 – 4:55

"Olhem o que a gente vai fazer: escrever a solução em português, antes de escrever em código.

Criar uma lista vazia. Para cada vaga das 120: se o nível for junior, e o salário for dois mil ou mais, guardar na minha lista. No fim, mostrar a lista.

Seis linhas. Nenhuma delas é JavaScript. E mesmo assim **o problema já está resolvido**.

Da linha 2 vai sair a repetição. Das linhas 3 e 4, a condicional. Traduzir isso para o computador agora é a parte fácil.

Se você não consegue explicar em português, você ainda não sabe o que vai programar. É por isso que tanta gente trava na frente do computador — não travou no código, travou antes dele."

FAÇA Aponte fisicamente para cada linha ao dizer de onde sai cada estrutura.

11 Variáveis, tipos e objeto

Primeira peça de código

4:55 – 6:10

"Primeira peça: **variável**. Variável é só um nome para um valor guardado. Aqui eu guardei o texto 'junior' num nome chamado meuNivel.

Segunda peça: **objeto**. Um objeto junta vários valores que descrevem a mesma coisa. Esta vaga tem título, empresa, nível e salário — tudo num pacote só.

E reparem nos tipos. O título está entre aspas: é texto, é uma **string**. O salário, 2200, está sem aspas: é **número**.

Isso importa muito. Se eu colocasse aspas no 2200, o computador ia tratar como texto — e texto não dá pra comparar por tamanho. Esse é o erro número um de quem está começando."

12 Array de objetos

Segunda interação com a turma

`6:10 – 7:10`

"Terceira peça: o **array**. Array é uma lista de coisas.

A gente não tem uma vaga, tem 120. Então coloca todas numa lista só. O `.length` me diz quantas tem: 120. E o colchete zero me dá a primeira da lista.

Deixa eu perguntar: se a primeira é `vagas[0]`, qual é a última das 120?"

(espere — e deixe alguém errar primeiro)

"É 119. A contagem começa no zero, então a última é sempre uma a menos. Todo mundo erra isso na primeira semana, e tudo bem. Mas a partir de hoje vocês já sabem."

FAÇA Interação nº 2. Deixar alguém errar é melhor que ninguém responder — o erro fixa mais.

13 Condicional

Ler código em voz alta como português

`7:10 – 8:25`

"Agora a peça que faz o computador **decidir**: o `if`. Se.

Leiam comigo em português: **se** o nível dessa vaga **for igual a** junior, mostra 'essa serve pra mim'. **Senão**, mostra 'essa não'.

Só isso. O `if` faz uma pergunta que só tem duas respostas: sim ou não.

Três sinais que vocês vão usar sempre: três iguais quer dizer 'é igual a'. Maior-igual quer dizer 'é maior ou igual a'. E o E-comercial dobrado quer dizer 'e' — ele exige as duas coisas ao mesmo tempo.

Nível junior **E** salário de dois mil pra cima. Se fosse 'ou', bastava uma das duas."

FAÇA Leia o código em voz alta traduzindo para português. É o hábito que mais destrava iniciante.

14 Repetição

O momento em que a mágica aparece

`8:25 – 9:40`

"Mas eu não quero testar uma vaga. Quero testar as 120. Pra isso existe o `for`.

Ele tem três partes: onde começa, até quando vai, e de quanto em quanto anda. Começa no zero, vai enquanto o `i` for menor que o tamanho da lista, e anda de um em um.

Dentro dele, o `if` que a gente acabou de ver. Se a vaga passa, entra na minha lista.

E aqui está o resultado: de 120, sobraram **24**.

Reparem no que aconteceu. Eu escrevi a regra **uma vez só**, e o computador aplicou 120 vezes. Se amanhã a lista virar dez mil vagas, este código não muda uma única linha."

15 Funções

Se estiver atrasado, encurte aqui — mas não pule

`9:40 – 11:00`

"Última peça: **função**. Função é um pedaço de código que ganha um nome. Você escreve uma vez e usa quantas vezes quiser.

Olhem o que tem dentro dela: é **exatamente o mesmo código** que a gente acabou de escrever. A lista vazia, o `for`, o `if`.

A diferença é uma só: o nível não está mais fixo lá dentro. Ele vem de fora, na hora em que eu chamo a função.

Então eu chamo buscarVagas com 'junior' e recebo 34. Chamo com 'estagio' e recebo 35. Mesmo código, respostas diferentes.

É pra isso que serve uma função."

16 O resultado rodando

A virada da aula

`11:00 – 11:45`

"E é isso rodando de verdade.

De 120 vagas, 24 são pra você. Com nome, empresa e salário.

Uma lista impossível de ler virou uma lista possível. Isso é lógica de programação — a primeira metade da nossa pergunta está resolvida.

(pausa de verdade — dois segundos)

"Agora vem a segunda metade. E é a parte que ninguém ensina na primeira aula, e que quebra todo mundo no primeiro trabalho em grupo."

FAÇA Se der, rode `node vagas.js` ao vivo no terminal em vez de mostrar o slide. Código executando vale muito em domínio técnico.

17 Anatomia de um projeto

Item do tema sorteado — nomeie em voz alta

`11:45 – 12:50`

"Antes de falar de equipe, uma pergunta: onde é que esse código mora?

Quem aqui já salvou um arquivo com nome tipo 'trabalho final 2'?"

(deixe rir)

"Pois é. Código solto numa pasta qualquer não é projeto.

Projeto tem estrutura: uma pasta com nome, o código separado dos dados, um `README` explicando o que é e como rodar, e um arquivo dizendo o que **não** deve ser versionado.

Esse README é a porta de entrada. Projeto sem README é casa sem placa.

Isso se chama **abertura de projeto**, e é o que permite outra pessoa — ou você mesmo daqui a três meses — abrir e entender em cinco minutos."

ATENÇÃO Diga "abertura de projeto" com todas as letras: é um dos quatro itens do tema que foi sorteado para você.

18 Os quatro comandos

Se estiver atrasado, este é o 2º a cortar

`12:50 – 13:40`

"E abrir um projeto são quatro comandos.

Criar a pasta e entrar nela. Gerar o arquivo de identidade do projeto. Começar a versionar. E dizer o que deve ser ignorado.

Vinte segundos.

E a partir desse `git init`, o terceiro comando, tudo que acontecer nessa pasta pode ser **gravado, comparado e desfeito**."

19 A dor do versionamento

O slide que ganha a turma

`13:40 – 14:20`

"Agora: por que versionar? Deixa eu mostrar o que acontece quando não se versiona.

(mostre a lista de arquivos)

"Quem nunca?"

(deixe rir de verdade — não corra)

"O problema não é preguiça. É que ninguém ensinou o jeito certo. E o jeito certo tem nome: **controle de versão**. Git existe pra isso nunca mais acontecer."

ATENÇÃO Riso na sala conta como interação. Não atropele o momento.

20 As três áreas do Git

Slide técnico central do bloco

14:20 – 15:30

"Git parece complicado, mas tem só **três lugares**. Se vocês entenderem este desenho, entendem todos os comandos.

Primeiro: **a sua pasta**. É onde você trabalha, onde edita os arquivos.

Segundo: **a prateleira**. É onde você separa o que quer salvar — porque nem tudo que você mexeu deve ser salvo junto.

Terceiro: **o histórico**. O que entra aqui fica gravado.

E os comandos são as setas entre eles. `git add` empurra da pasta pra prateleira. `git commit` grava da prateleira no histórico. E `git push` manda o histórico pro GitHub, que é a nuvem.

Quase todo erro de iniciante em Git é confundir esses três lugares."

FAÇA Use as mãos: três posições no ar, da esquerda para a direita. O gesto fixa o modelo melhor que o slide.

21 Comandos essenciais

Commit como save point

15:30 – 16:35

"Na prática são quatro comandos. `git status`: o que mudou? `git add`: manda pra prateleira. `git commit`: grava, com uma mensagem dizendo o que você fez. `git log`: mostra tudo que já foi gravado.

E olhem o histórico aqui embaixo, lendo de baixo pra cima: abri o projeto, carreguei as vagas, filtrei por nível. Dá pra contar a história inteira do projeto lendo **só as mensagens**.

Por isso mensagem importa. 'Ajustes' não diz nada. 'Filtra vagas por nível júnior' diz tudo.

E esse histórico é a prova do seu raciocínio — é literalmente o que um recrutador vai ler no seu GitHub."

BÔNUS Aponte para a barra no rodapé do slide: "essa linha embaixo da tela é um histórico de commits — cada slide desta aula foi um."

22 Branch, Pull Request e merge

Fecha a 2ª metade do problema gerador

16:35 – 17:45

"Agora sim, a segunda metade da nossa pergunta: como trabalhar junto sem se atrapalhar.

Imaginem que a Ana vai fazer o filtro e o Bruno vai fazer a ordenação. Ao mesmo tempo, no mesmo projeto.

Cada um cria uma **branch** — uma linha própria de trabalho. A Ana trabalha na dela, o Bruno na dele, e nenhum dos dois encosta no trabalho do outro.

Quando termina, cada um abre um **Pull Request**: um pedido de revisão. O squad lê o código, comenta, sugere.

Aprovado, vem o **merge**: o Git junta os dois trabalhos, sem apagar nada de ninguém.

E essa é a resposta para a segunda metade do nosso **problema gerador**."

> **ATENÇÃO** Volte ao problema gerador em voz alta aqui. Fechar o ciclo na frente da banca é o que carrega os 10 pontos de metodologia.

23 GitHub como portfólio

O momento emocional — fale devagar

17:45 – 18:35

"E aí, lembram das 120 vagas com que a gente começou?

Quando vocês forem se candidatar a uma delas, a primeira coisa que vão pedir é o link do seu GitHub.

O repositório que vocês construírem aqui **é a resposta**. README explicando o problema, histórico de commits contando a história, Pull Requests revisados pelo squad, e código que roda.

Isso não é exercício de aula. É a primeira peça do portfólio de vocês. E vale mais que qualquer certificado impresso."

> **FAÇA** Diminua o ritmo. Olhe para a banca. Aqui o círculo da aula se fecha.

24 Entrega e avaliação

Responde antes de perguntarem

18:35 – 19:20

"Sobre a entrega — e quero ser bem claro aqui.

Hoje ninguém vai sair daqui com tudo isso pronto, e nem é pra sair. Hoje vocês saem com três coisas: conta criada no GitHub, a pasta do projeto criada, e o primeiro commit feito.

O resto — o repositório completo, as branches, os Pull Requests — a gente constrói ao longo dos dez encontros.

Hoje a gente percorreu o caminho inteiro de uma vez, pra vocês verem aonde isso chega. Nos próximos encontros a gente volta em cada parada com calma.

E a avaliação não é prova: é o próprio repositório de vocês. Reparem que 40% da nota é sobre **como** vocês trabalharam juntos — histórico e colaboração — não só sobre o código funcionar."

ATENÇÃO Este slide desarma a pergunta "não é conteúdo demais para uma aula?" antes de ela ser feita.

25 Fechamento

Encerre dizendo que encerrou no tempo

19:20 – 20:00

"Fechando.

Lógica resolve. Versionamento preserva.

Vocês pegaram uma lista impossível de ler e ensinaram o computador a lê-la. E aprenderam a construir isso com outras pessoas sem perder nada no caminho.

Tudo que está escrito aqui embaixo a gente viu hoje, de passagem. Nos próximos nove encontros, cada um desses vira uma aula inteira.

No próximo encontro, cada squad apresenta seu repositório e revisa o do outro.

Obrigado."

FAÇA Pare o cronômetro e diga, virado para a banca: **"encerro a exposição no tempo previsto"**. Gestão do tempo é uma linha da ficha — não deixe implícito.

Parte 3 — Arguição

10 minutos · Art. 40, III · o tempo agora é da banca

Q&A **Como se portar** — 10 min

Dois slides de apoio ao final da apresentação

Se souber: responda em 30 a 60 segundos e pare. Resposta longa demais parece insegurança.

Se não souber: "Isso eu não domino a fundo, mas o caminho que eu seguiria é ___." Honestidade com raciocínio pontua mais que improviso errado, e a banca percebe a diferença na hora.

Se perguntarem sobre as 40 horas: é a deixa para o site da trilha. Aí sim você abre e mostra os dez encontros.

Se perguntarem se não é conteúdo demais: você já respondeu no slide 24 — reforce que é a aula panorâmica de abertura.

FAÇA Aperte `0` para abrir o mapa e ir direto a qualquer slide que a banca citar. Os dois últimos slides trazem 12 perguntas prováveis já respondidas.